

Name: Gargi P. Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE (ICB)

SSE ASSIGNMENT 2

Q.1 Which test is performed after Unit Testing. Explain in detail.

- This type of testing is performed by developers before the setup is handed over in the testing team to formally execute the test cases.
- Unit testing is performed by the respective developers on the individual units of source code assigned areas.
- The developers use test data that is different from the test data of the quality assurance team.
- The goal of unit testing is to isolate each part of the program and show that individual parts are correct in terms of requirements and functionality.

Limitations of Unit Testing

- Testing cannot catch each and every bug in an application.
- It is impossible to evaluate every execution path in every software application. The same is the case with unit testing.
- There is a limit to the number of scenarios and test data that a developer can use to verify a source code.
- After having exhausted all the options, there is no choice but to stop unit testing and merge the code segment with other units.

Q.2 Write the test cases for login page.

⇒ Test cases are a set of conditions or input used to verify whether a system or component works as expected.
 For a login page, testing ensures secure & valid user authentication, usability and error handling.

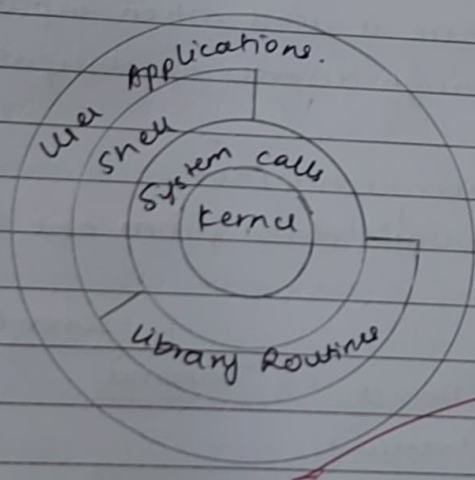
Test case ID	Test case objective	Input data	Expected result	Actual result	Status
TC1	Check cursor position at email or mobile number field.	Click on email or mobile number field.	Cursor should be placed on the field	Placed the cursor on the field	Pass
TC2	Check cursor position at password field	Click on password field	Cursor should be placed on the password field	Placed the cursor on the password field.	Pass
TC3	Check the continue button	It should redirect to password page	It redirected to the password page	It redirected to the password page	Pass
TC4	Readability of font	Try to read content on login page	Contents should be readable	Contents are readable	Pass
TC5	Testing of	check the spelling of login	Login Spelling should	Spelling of Login	Pass

Q-3.

Discuss various architecture design.

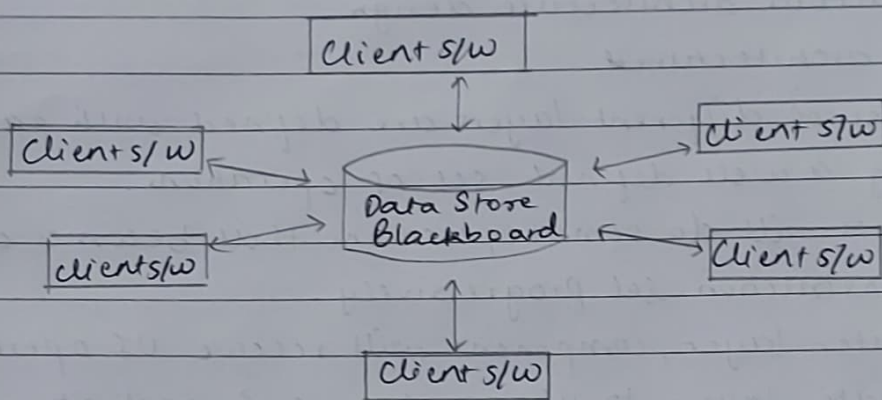
⇒ ① Layered architecture

- A number of different layers are defined with each layer performing a well defined set of operations.
- Each layer will do some operations that become closer to machine instruction set progressively.
- At the outer layer, component will receive UI operations.
- Intermediate layers to utility services & application software functions.



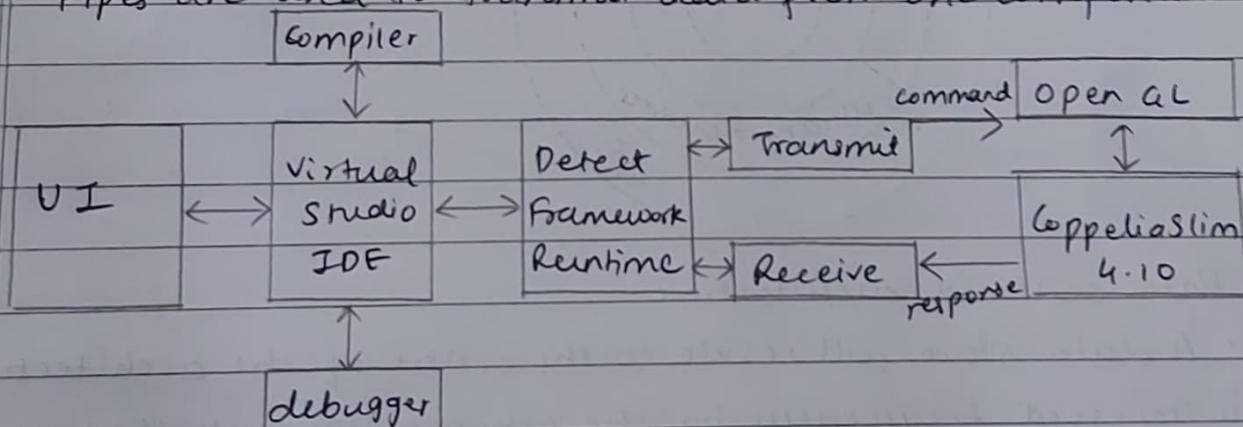
② Data centered architecture

- A data store will reside to the centre of the architecture & is accessed frequently by the other components that update, add, delete or modify the data.
- The client software accesses a central repository.
- Variation of this approach are used to transform the repository into a blackboard when data related to client or data of interest for the client.
- Data can be passed among clients using blackboard mechanism.



③ Data Flow architecture.

- This kind of architecture is used when input data is transformed into output data through a series of computational manipulative components.
- This is a pipe and filter architecture.
- Pipes are used to transmit data from one component to next.



Q.4. Explain the importance of cohesion and coupling in secure software engineering and their types.

⇒ ① Cohesion refers to how closely the operations within a single module or class are related to each other. It measures the degree to which the elements within a module work together to achieve a single, well-defined task. Following are the importance of cohesion:

- Improved readability & Maintainability: High cohesion often leads to modules that are easier to understand, modify and test.
- Reduced risk of security Flaws: When a module is tightly focused on a single responsibility, it's less likely to introduce unattended interactions.
- Easier updates and patching. A module with high cohesion can be more easily updated or patched without affecting other areas of the system.

② Coupling Types of Cohesion:

- 1) Functional Cohesion: different elements cooperate to achieve a single function.
- 2) Sequential cohesion: If element of a module form the components of the sequence, where the output from one component of the sequence is input to the next.
- 3) Communicational Cohesion: A module is said to have communicational cohesion if all tasks of the module refer to or update the same data structure.
- 4) Procedural Cohesion: If set of purpose of the module are all part of a procedure.
- 5) Temporal Cohesion: Includes ~~functions~~ that are associated by the fact that all methods must be executed.
- 6) Logical Cohesion: If elements of a module perform a similar operation.
- 7) Coincidental Cohesion: A module is said to have coincidental cohesion if it performs a set of tasks that are associated with each other very loosely.

② Coupling: Coupling refers to the degree of dependence between different modules or components in a software system.

Importance

Types of Coupling:

- Isolation of Security Risks: Ensures that failures don't cascade into others.
- Easier Security audits and testing: Easier to audit, test and validate with loose coupling.
- Simplified Maintenance: Updates and changes can be made to one module without affecting others.

Types of Coupling

- 1) No direct Coupling: Modules are subordinates to different modules. No direct coupling is present.
- 2) Data Coupling: Data is passed to another module.
- 3) Stamp Coupling: 2 modules are stamp coupled if they communicate using composite data items.
- 4) Control Coupling: Data from one module is used to direct the structure of instruction execution in another.
- 5) External Coupling: 2 modules share an externally imposed data format.
- 6) Common Coupling: They share information through some global data items.

5. Explore pointer based vulnerability with example.

- ⇒ Pointer based vulnerabilities are crucial security flaws that occur in programs written in languages like C++, where direct memory access is permitted. These vulnerabilities arise when pointer variables that store memory addresses are misused or improperly managed. Such vulnerabilities can lead to system crashes, data corruptions or even allow attackers to execute arbitrary code.

• Buffer Overflow: It occurs when a program writes more data into a memory buffer than it can hold. Since memory buffers are often allocated with a fixed size, any excess data can overcome adjacent memory. This may include sensitive data, or control information like return addresses on the call stack. Risk: An attacker could overwrite the return address of a function. Also there can be unauthorized access.

• Null Pointer Access: ^{Reference:} When an attempt to access or modify memory using a pointer that has not been initialized, it can cause a crash. This is a common cause of segmentation faults in programs.

Risks: While often resulting in a crash, if combined with other vulnerabilities this can be a part of a larger exploit.